

```
1  # Java 基础知识讲解课件
2
3  > 适合对象: Java 零基础 / 初学者
4  > 学习目标: 理解 Java 的基本语法、面向对象思想、常用类、异常、集合和基础开发流程。
5
6  ---
7
8  ## 目录
9
10 1. Java 简介
11 2. Java 开发环境
12 3. 第一个 Java 程序
13 4. Java 基本语法
14 5. 变量与数据类型
15 6. 运算符
16 7. 流程控制
17 8. 数组
18 9. 方法
19 10. 面向对象基础
20 11. 封装、继承、多态
21 12. 抽象类与接口
22 13. 常用类
23 14. 异常处理
24 15. 集合框架
25 16. Java 程序开发规范
26 17. 综合练习
27
28 ---
29
30 # 一、Java 简介
31
32 ## 1. 什么是 Java?
33
34 Java 是一种面向对象的编程语言，广泛应用于：
35
36 - 后端开发
37 - Android 开发
38 - 企业级系统开发
39 - 大数据开发
40 - 桌面应用开发
41 - 分布式系统开发
42
43 Java 的核心特点是：
44
45 > 一次编写，到处运行。
46
47 也就是说，只要电脑安装了 Java 运行环境，同一个 Java 程序就可以在不同系统上运行，比如
48 windows、Linux、macOS。
49
50 ---
51
52 ## 2. Java 的特点
53
54 ### 1. 面向对象
```

```
54
55  Java 是面向对象语言，程序主要由“类”和“对象”组成。
56
57  例如：
58
59  ```java
60  class Student {
61      String name;
62      int age;
63  }
```

Fence 1

这里的 `Student` 就是一个类，用来描述学生这种对象。

0.1 2. 跨平台

Java 程序不是直接运行在操作系统上的，而是运行在 JVM 上。

```
1  Java 源代码 -> 字节码文件 -> JVM 运行
```

Fence 2

源代码文件：

```
1  HelloWorld.java
```

Fence 3

编译后生成：

```
1  HelloWorld.class
```

Fence 4

JVM 负责在不同操作系统上运行 `.class` 文件。

0.1 3. 安全性高

Java 不允许程序直接操作内存地址，减少了很多危险操作。

0.1 4. 自动内存管理

Java 有垃圾回收机制，程序员不需要手动释放内存。

二、Java 开发环境

1. 1. JDK、JRE、JVM 的区别

1.1 JVM

JVM 全称是 Java Virtual Machine，中文叫 Java 虚拟机。

作用：

运行 Java 字节码文件。

1.2 JRE

JRE 全称是 Java Runtime Environment，中文叫 Java 运行环境。

它包含：

1 | JRE = JVM + Java 核心类库

Fence 5

作用：

用来运行 Java 程序。

1.3 JDK

JDK 全称是 Java Development Kit，中文叫 Java 开发工具包。

它包含：

1 | JDK = JRE + 开发工具

Fence 6

常用开发工具有：

- `javac`：编译 Java 文件
 - `java`：运行 Java 程序
 - `javadoc`：生成 Java 文档
-

2. 2. Java 程序运行流程

```
1  编写源代码 HelloWorld.java
2      ↓
3  使用 javac 编译
4      ↓
5  生成 HelloWorld.class
6      ↓
7  使用 java 运行
8      ↓
9  JVM 执行程序
```

Fence 7

三、第一个 Java 程序

1. 1. HelloWorld 程序

```
1 public class HelloWorld {
2     public static void main(String[] args) {
3         System.out.println("Hello, Java!");
4     }
5 }
```

Fence 8

2. 2. 代码解释

2.1 public class HelloWorld

```
1 public class HelloWorld
```

Fence 9

表示定义了一个公共类，类名叫 `HelloWorld`。

注意：

Java 文件名必须和 `public` 修饰的类名一致。

所以文件名必须是：

```
1 HelloWorld.java
```

Fence 10

2.2 main 方法

```
1 public static void main(String[] args)
```

Fence 11

这是 Java 程序的入口。

程序运行时，会从 `main` 方法开始执行。

2.3 输出语句

```
1 System.out.println("Hello, Java!");
```

Fence 12

作用是在控制台输出一行文字。

输出结果：

```
1 Hello, Java!
```

Fence 13

四、Java 基本语法

1. 1. Java 代码基本结构

```
1 public class 类名 {  
2     public static void main(String[] args) {  
3         // 程序代码  
4     }  
5 }
```

Fence 14

2. 2. Java 语句以分号结尾

```
1 int age = 18;  
2 System.out.println(age);
```

Fence 15

每条完整语句后面都要写分号 `;`。

3. 3. Java 严格区分大小写

下面两个变量是不同的：

```
1 int age = 18;  
2 int Age = 20;
```

Fence 16

`age` 和 `Age` 不是同一个变量。

4. 4. 注释

注释是给人看的说明文字，不会被程序执行。

4.1 单行注释

```
1 // 这是单行注释
```

Fence 17

4.2 多行注释

```
1 /*  
2 这是多行注释  
3 可以写多行内容  
4 */
```

Fence 18

4.3 文档注释

```
1  /**
2   * 这是文档注释
3   * 通常用来说明类或方法
4   */
```

Fence 19

五、变量与数据类型

1. 1. 什么是变量？

变量可以理解为一个“存放数据的盒子”。

例如：

```
1  int age = 18;
```

Fence 20

含义：

- int：数据类型
- age：变量名
- 18：变量中存放的值

2. 2. 变量的定义格式

```
1  数据类型 变量名 = 值;
```

Fence 21

例如：

```
1  int number = 100;
2  double price = 9.9;
3  char gender = '男';
4  boolean flag = true;
5  String name = "张三";
```

Fence 22

3. 3. Java 基本数据类型

Java 有 8 种基本数据类型。

类型	说明	示例
byte	字节型整数	byte a = 10;
short	短整型	short b = 100;
int	整型	int c = 1000;
long	长整型	long d = 10000L;
float	单精度浮点型	float e = 3.14F;
double	双精度浮点型	double f = 3.14;
char	字符型	char g = 'A';

类型	说明	示例
boolean	布尔型	<code>boolean h = true;</code>

Table 1

4. 4. 引用数据类型

除了基本数据类型，Java 中还有引用数据类型。

常见引用类型：

```
1 String name = "张三";
2 int[] arr = {1, 2, 3};
3 Student stu = new Student();
```

Fence 23

常见引用类型包括：

- 类
- 数组
- 接口
- 字符串

5. 5. String 字符串

`String` 用来表示字符串。

```
1 String name = "李四";
2 System.out.println(name);
```

Fence 24

注意：

```
1 char c = 'A';
2 String s = "A";
```

Fence 25

区别：

- `char` 用单引号，只能存一个字符
- `String` 用双引号，可以存多个字符

6. 6. 数据类型转换

6.1 自动类型转换

小范围类型可以自动转换成大范围类型。

```
1  int a = 10;
2  double b = a;
3  System.out.println(b);
```

Fence 26

输出：

```
1  10.0
```

Fence 27

6.2 强制类型转换

大范围类型转小范围类型，需要强制转换。

```
1  double a = 3.14;
2  int b = (int) a;
3  System.out.println(b);
```

Fence 28

输出：

```
1  3
```

Fence 29

注意：

强制类型转换可能会造成数据丢失。

六、运算符

1. 1. 算术运算符

运算符	作用	示例
+	加法	a + b
-	减法	a - b
*	乘法	a * b
/	除法	a / b
%	取余	a % b

Table 2

示例：

```
1  int a = 10;
2  int b = 3;
3
4  System.out.println(a + b);
5  System.out.println(a - b);
6  System.out.println(a * b);
7  System.out.println(a / b);
8  System.out.println(a % b);
```

Fence 30

输出：

```
1  13
2  7
3  30
4  3
5  1
```

Fence 31

2. 2. 自增自减运算符

```
1  int a = 10;
2  a++;
3  System.out.println(a);
```

Fence 32

输出：

```
1  11
```

Fence 33

2.1 前置自增

```
1 | int a = 10;
2 | int b = ++a;
```

Fence 34

先自增，再赋值。

结果：

```
1 | a = 11
2 | b = 11
```

Fence 35

2.2 后置自增

```
1 | int a = 10;
2 | int b = a++;
```

Fence 36

先赋值，再自增。

结果：

```
1 | a = 11
2 | b = 10
```

Fence 37

3. 3. 关系运算符

关系运算符的结果是 `boolean` 类型。

运算符	说明
>	大于
<	小于
>=	大于等于
<=	小于等于
==	等于
!=	不等于

Table 3

示例：

```
1  int a = 10;
2  int b = 20;
3
4  System.out.println(a > b);
5  System.out.println(a == b);
6  System.out.println(a != b);
```

Fence 38

4. 4. 逻辑运算符

运算符	说明
&&	与, 并且
,	
!	非, 取反

Table 4

示例:

```
1  int age = 20;
2
3  System.out.println(age > 18 && age < 60);
4  System.out.println(age < 18 || age > 60);
5  System.out.println(!(age > 18));
```

Fence 39

5. 5. 赋值运算符

```
1  int a = 10;
2
3  a += 5; // 相当于 a = a + 5
4  a -= 2; // 相当于 a = a - 2
5  a *= 3; // 相当于 a = a * 3
6  a /= 2; // 相当于 a = a / 2
7  a %= 4; // 相当于 a = a % 4
```

Fence 40

七、流程控制

1. 1. 顺序结构

程序默认从上到下依次执行。

```
1 System.out.println("第一步");
2 System.out.println("第二步");
3 System.out.println("第三步");
```

Fence 41

2. 2. 分支结构 if

2.1 单分支

```
1 int age = 20;
2
3 if (age >= 18) {
4     System.out.println("成年人");
5 }
```

Fence 42

2.2 双分支

```
1 int age = 16;
2
3 if (age >= 18) {
4     System.out.println("成年人");
5 } else {
6     System.out.println("未成年人");
7 }
```

Fence 43

2.3 多分支

```
1 int score = 85;
2
3 if (score >= 90) {
4     System.out.println("优秀");
5 } else if (score >= 80) {
6     System.out.println("良好");
7 } else if (score >= 60) {
8     System.out.println("及格");
9 } else {
10    System.out.println("不及格");
11 }
```

Fence 44

3. 3. switch 语句

适合对固定值进行判断。

```
1  int week = 3;
2
3  switch (week) {
4      case 1:
5          System.out.println("星期一");
6          break;
7      case 2:
8          System.out.println("星期二");
9          break;
10     case 3:
11         System.out.println("星期三");
12         break;
13     default:
14         System.out.println("输入错误");
15         break;
16 }
```

Fence 45

注意：

`break` 用来结束当前 case，防止继续向下执行。

4. 4. 循环结构 for

```
1  for (int i = 1; i <= 5; i++) {
2      System.out.println("第" + i + "次循环");
3  }
```

Fence 46

执行流程：

```
1  1. 初始化变量 i = 1
2  2. 判断 i <= 5
3  3. 执行循环体
4  4. i++
5  5. 回到第 2 步
```

Fence 47

5. 5. while 循环

```
1  int i = 1;
2
3  while (i <= 5) {
4      System.out.println(i);
5      i++;
6  }
```

Fence 48

适合不知道具体循环次数，但知道循环条件的情况。

6.6. do...while 循环

```
1  int i = 1;
2
3  do {
4      System.out.println(i);
5      i++;
6  } while (i <= 5);
```

Fence 49

特点：

do...while 至少会执行一次循环体。

7.7. break 和 continue

7.1 break

结束整个循环。

```
1  for (int i = 1; i <= 5; i++) {
2      if (i == 3) {
3          break;
4      }
5      System.out.println(i);
6  }
```

Fence 50

输出：

```
1  1
2  2
```

Fence 51

7.2 continue

跳过本次循环，继续下一次循环。

```
1  for (int i = 1; i <= 5; i++) {
2      if (i == 3) {
3          continue;
4      }
5      System.out.println(i);
6  }
```

Fence 52

输出：

1	1
2	2
3	4
4	5

八、数组

1. 1. 什么是数组？

数组是用来存储多个相同类型数据的容器。

例如：

```
1 | int[] scores = {90, 80, 70};
```

Fence 54

这个数组中存放了 3 个整数。

2. 2. 数组定义方式

2.1 静态初始化

```
1 | int[] arr = {1, 2, 3, 4, 5};
```

Fence 55

2.2 动态初始化

```
1 | int[] arr = new int[5];
```

Fence 56

表示创建一个长度为 5 的整数数组。

3. 3. 访问数组元素

数组下标从 0 开始。

```
1 | int[] arr = {10, 20, 30};  
2 |  
3 | System.out.println(arr[0]);  
4 | System.out.println(arr[1]);  
5 | System.out.println(arr[2]);
```

Fence 57

输出：

```
1 | 10  
2 | 20  
3 | 30
```

Fence 58

4. 4. 修改数组元素

```
1  int[] arr = {10, 20, 30};
2  arr[1] = 100;
3
4  System.out.println(arr[1]);
```

Fence 59

输出:

```
1  100
```

Fence 60

5. 5. 数组长度

```
1  int[] arr = {10, 20, 30};
2
3  System.out.println(arr.length);
```

Fence 61

输出:

```
1  3
```

Fence 62

6. 6. 遍历数组

```
1  int[] arr = {10, 20, 30};
2
3  for (int i = 0; i < arr.length; i++) {
4      System.out.println(arr[i]);
5  }
```

Fence 63

7. 7. 增强 for 循环

```
1  int[] arr = {10, 20, 30};
2
3  for (int num : arr) {
4      System.out.println(num);
5  }
```

Fence 64

适合只读取数组元素，不修改下标。

九、方法

1. 1. 什么是方法？

方法是一段可以重复使用的代码。

例如：

```
1 public static void sayHello() {  
2     System.out.println("你好, Java");  
3 }
```

Fence 65

调用方法：

```
1 sayHello();
```

Fence 66

2. 2. 方法定义格式

```
1 修饰符 返回值类型 方法名(参数列表) {  
2      方法体;  
3      return 返回值;  
4 }
```

Fence 67

3. 3. 无参数无返回值方法

```
1 public static void printHello() {  
2     System.out.println("Hello");  
3 }
```

Fence 68

调用：

```
1 printHello();
```

Fence 69

4. 4. 有参数无返回值方法

```
1 public static void printName(String name) {  
2     System.out.println("姓名: " + name);  
3 }
```

Fence 70

调用：

```
1 printName("张三");
```

5.5. 有参数有返回值方法

```
1 public static int add(int a, int b) {  
2     return a + b;  
3 }
```

Fence 72

调用：

```
1 int result = add(10, 20);  
2 System.out.println(result);
```

Fence 73

6.6. return 关键字

`return` 有两个作用：

1. 返回结果
2. 结束方法

示例：

```
1 public static int getMax(int a, int b) {  
2     if (a > b) {  
3         return a;  
4     }  
5     return b;  
6 }
```

Fence 74

7.7. 方法重载

方法重载指的是：

在同一个类中，方法名相同，但参数列表不同。

示例：

```
1 public static int add(int a, int b) {  
2     return a + b;  
3 }  
4  
5 public static double add(double a, double b) {  
6     return a + b;  
7 }  
8  
9 public static int add(int a, int b, int c) {  
10    return a + b + c;  
11 }
```

Fence 75

判断方法重载的依据：

- 参数个数不同
- 参数类型不同
- 参数顺序不同

注意：

只改变返回值类型，不构成方法重载。

十、面向对象基础

1. 1. 什么是面向对象？

面向对象是一种编程思想。

它把现实世界中的事物抽象成程序中的对象。

例如现实中的学生有：

- 姓名
- 年龄
- 学号
- 学习行为

在 Java 中可以写成：

```
1  class Student {
2      String name;
3      int age;
4      String id;
5
6      public void study() {
7          System.out.println(name + "正在学习");
8      }
9  }
```

Fence 76

2. 2. 类和对象

2.1 类

类是对象的模板。

例如：

```
1  class Student {
2      String name;
3      int age;
4  }
```

Fence 77

2.2 对象

对象是类的具体实例。

```
1  Student stu = new Student();
2  stu.name = "张三";
3  stu.age = 18;
```

Fence 78

3. 3. 创建对象

```
1 | 类名 对象名 = new 类名();
```

Fence 79

示例:

```
1 | student stu = new Student();
```

Fence 80

4. 4. 使用对象属性和方法

```
1 | class Student {  
2 |     String name;  
3 |     int age;  
4 |  
5 |     public void study() {  
6 |         System.out.println(name + "正在学习");  
7 |     }  
8 | }  
9 |  
10 | public class Test {  
11 |     public static void main(String[] args) {  
12 |         Student stu = new Student();  
13 |         stu.name = "李四";  
14 |         stu.age = 20;  
15 |         stu.study();  
16 |     }  
17 | }
```

Fence 81

输出:

```
1 | 李四正在学习
```

Fence 82

5. 5. 成员变量和局部变量

5.1 成员变量

定义在类中、方法外。

```
1 | class Student {  
2 |     String name;  
3 |     int age;  
4 | }
```

Fence 83

5.2 局部变量

定义在方法中。

```
1 public void show() {
2     int num = 10;
3 }
```

Fence 84

6. 6. 成员变量和局部变量区别

区别	成员变量	局部变量
定义位置	类中方法外	方法中
默认值	有默认值	没有默认值
作用范围	整个对象	当前方法
生命周期	随对象创建和销毁	随方法调用和结束

Table 5

十一、封装、继承、多态

1. 封装

1.1 什么是封装？

封装就是把对象的属性隐藏起来，不允许外部直接访问，而是通过方法来访问。

这样可以提高安全性。

2. 1.2 private 关键字

```
1  class Student {  
2      private String name;  
3      private int age;  
4  }
```

Fence 85

`private` 修饰的成员只能在当前类内部访问。

3. 1.3 getter 和 setter 方法

```
1  class Student {  
2      private String name;  
3      private int age;  
4  
5      public void setName(String name) {  
6          this.name = name;  
7      }  
8  
9      public String getName() {  
10         return name;  
11     }  
12  
13     public void setAge(int age) {  
14         if (age >= 0 && age <= 120) {  
15             this.age = age;  
16         } else {  
17             System.out.println("年龄不合法");  
18         }  
19     }  
20  
21     public int getAge() {  
22         return age;  
23     }  
24 }
```

Fence 86

使用：

```
1 Student stu = new Student();
2 stu.setName("张三");
3 stu.setAge(18);
4
5 System.out.println(stu.getName());
6 System.out.println(stu.getAge());
```

Fence 87

4. 1.4 this 关键字

`this` 表示当前对象。

```
1 this.name = name;
```

Fence 88

左边的 `this.name` 是成员变量，右边的 `name` 是方法参数。

2. 继承

1. 2.1 什么是继承？

继承表示一个类可以拥有另一个类的属性和方法。

被继承的类叫父类，也叫超类。

继承别人的类叫子类。

2. 2.2 继承格式

```
1 class 子类 extends 父类 {  
2 }
```

Fence 89

示例：

```
1 class Animal {  
2     public void eat() {  
3         System.out.println("动物吃东西");  
4     }  
5 }  
6  
7 class Dog extends Animal {  
8     public void bark() {  
9         System.out.println("狗叫");  
10    }  
11 }
```

Fence 90

使用：

```
1 Dog dog = new Dog();  
2 dog.eat();  
3 dog.bark();
```

Fence 91

输出：

```
1 动物吃东西  
2 狗叫
```

Fence 92

3. 2.3 继承的好处

- 提高代码复用性
- 提高代码扩展性
- 让类与类之间产生关系

4. 2.4 方法重写

子类可以重写父类的方法。

```
1  class Animal {
2      public void eat() {
3          System.out.println("动物吃东西");
4      }
5  }
6
7  class Dog extends Animal {
8      @Override
9      public void eat() {
10         System.out.println("狗吃骨头");
11     }
12 }
```

Fence 93

5. 2.5 super 关键字

`super` 用来访问父类成员。

```
1  class Animal {
2      String name = "动物";
3  }
4
5  class Dog extends Animal {
6      String name = "狗";
7
8      public void show() {
9          System.out.println(super.name);
10         System.out.println(this.name);
11     }
12 }
```

Fence 94

3. 多态

1. 3.1 什么是多态?

多态指的是：

同一个对象，在不同时刻表现出不同形态。

常见格式：

```
1 父类类型 对象名 = new 子类类型();
```

Fence 95

示例：

```
1  Animal animal = new Dog();
```

Fence 96

2. 3.2 多态示例

```
1  class Animal {
2      public void eat() {
3          System.out.println("动物吃东西");
4      }
5  }
6
7  class Dog extends Animal {
8      @Override
9      public void eat() {
10         System.out.println("狗吃骨头");
11     }
12 }
13
14 public class Test {
15     public static void main(String[] args) {
16         Animal animal = new Dog();
17         animal.eat();
18     }
19 }
```

Fence 97

输出：

```
1  狗吃骨头
```

Fence 98

3. 3.3 多态的好处

- 提高程序扩展性
- 降低类之间的耦合

- 让代码更加灵活

4.3.4 向上转型和向下转型

4.1 向上转型

```
1 Animal animal = new Dog();
```

Fence 99

子类对象赋值给父类引用。

4.2 向下转型

```
1 Dog dog = (Dog) animal;
```

Fence 100

父类引用转回子类类型。

注意：

向下转型有风险，可能出现类型转换异常。

可以使用 `instanceof` 判断：

```
1 if (animal instanceof Dog) {  
2     Dog dog = (Dog) animal;  
3 }
```

Fence 101

十二、抽象类与接口

1. 抽象类

1.1.1 什么是抽象类？

如果一个类不能描述具体对象，只能作为父类使用，可以定义为抽象类。

使用 `abstract` 修饰。

```
1 abstract class Animal {  
2 }
```

Fence 102

2. 1.2 抽象方法

抽象方法只有方法声明，没有方法体。

```
1 abstract class Animal {  
2     public abstract void eat();  
3 }
```

Fence 103

3. 1.3 抽象类特点

- 抽象类不能直接创建对象
- 抽象类可以有普通方法
- 抽象类可以有成员变量
- 子类继承抽象类后，必须重写抽象方法，除非子类也是抽象类

4. 1.4 示例

```
1 abstract class Animal {  
2     public abstract void eat();  
3  
4     public void sleep() {  
5         System.out.println("睡觉");  
6     }  
7 }  
8  
9 class Dog extends Animal {  
10     @Override  
11     public void eat() {  
12         System.out.println("狗吃骨头");  
13     }  
14 }
```

Fence 104

使用：

```
1  Animal animal = new Dog();  
2  animal.eat();  
3  animal.sleep();
```

Fence 105

2. 接口

1. 2.1 什么是接口?

接口是一种规则或规范。

使用 `interface` 定义。

```
1 interface Flyable {  
2     void fly();  
3 }
```

Fence 106

2. 2.2 实现接口

类通过 `implements` 实现接口。

```
1 class Bird implements Flyable {  
2     @Override  
3     public void fly() {  
4         System.out.println("鸟会飞");  
5     }  
6 }
```

Fence 107

3. 2.3 接口特点

- 接口不能直接创建对象
- 接口中的方法默认是公共抽象方法
- 接口中的变量默认是常量
- 一个类可以实现多个接口

4. 2.4 示例

```
1 interface Swim {  
2     void swim();  
3 }  
4  
5 interface Run {  
6     void run();  
7 }  
8  
9 class Person implements Swim, Run {  
10     @Override  
11     public void swim() {  
12         System.out.println("人会游泳");  
13     }  
14 }
```

```
15      @Override
16      public void run() {
17          System.out.println("人会跑步");
18      }
19  }
```

Fence 108

5. 2.5 抽象类和接口区别

对比项	抽象类	接口
关键字	<code>abstract class</code>	<code>interface</code>
继承方式	<code>extends</code>	<code>implements</code>
多继承	类只能单继承	一个类可实现多个接口
成员变量	可以有普通变量	默认是常量
方法	可以有普通方法和抽象方法	主要是抽象方法
适用场景	表示“是什么”	表示“能做什么”

Table 6

十三、常用类

1. 1. Object 类

`Object` 是所有类的父类。

常用方法：

```
1 toString()
2 equals()
3 hashCode()
```

Fence 109

2. 2. toString 方法

默认输出对象地址。

```
1 Student stu = new Student();
2 System.out.println(stu);
```

Fence 110

通常需要重写：

```
1 class Student {
2     private String name;
3     private int age;
4
5     @Override
6     public String toString() {
7         return "Student{name='" + name + "', age=" + age + "}";
8     }
9 }
```

Fence 111

3. 3. equals 方法

`equals` 用来比较对象内容。

默认情况下，`equals` 比较地址。

通常需要重写：

```
1 class Student {
2     private String name;
3     private int age;
4
5     @Override
6     public boolean equals(Object obj) {
7         if (this == obj) {
8             return true;
9         }
10    }
```

```
10
11         if (obj == null || this.getClass() != obj.getClass()) {
12             return false;
13         }
14
15         Student student = (Student) obj;
16         return age == student.age && name.equals(student.name);
17     }
18 }
```

Fence 112

4. 4. String 类

`String` 表示字符串。

常用方法：

方法	作用
<code>length()</code>	获取字符串长度
<code>charAt(index)</code>	获取指定位置字符
<code>substring()</code>	截取字符串
<code>contains()</code>	判断是否包含
<code>equals()</code>	判断内容是否相等
<code>split()</code>	分割字符串
<code>replace()</code>	替换字符串

Table 7

示例：

```
1   String str = "Hello Java";
2
3   System.out.println(str.length());
4   System.out.println(str.charAt(0));
5   System.out.println(str.contains("Java"));
6   System.out.println(str.replace("Java", "World"));
```

Fence 113

5. 5. StringBuilder 类

`String` 字符串内容不可变，如果频繁拼接字符串，推荐使用 `StringBuilder`。


```
1  StringBuilder sb = new StringBuilder();
2
3  sb.append("Hello");
4  sb.append(" ");
5  sb.append("Java");
6
7  System.out.println(sb.toString());
```

Fence 114

输出:

```
1  Hello Java
```

Fence 115

6.6. Math 类

`Math` 用于数学运算。

```
1  System.out.println(Math.abs(-10));
2  System.out.println(Math.max(10, 20));
3  System.out.println(Math.min(10, 20));
4  System.out.println(Math.round(3.6));
5  System.out.println(Math.random());
```

Fence 116

7.7. Scanner 类

`Scanner` 用来接收键盘输入。

```
1  import java.util.Scanner;
2
3  public class Test {
4      public static void main(String[] args) {
5          Scanner scanner = new Scanner(System.in);
6
7          System.out.println("请输入姓名: ");
8          String name = scanner.next();
9
10         System.out.println("请输入年龄: ");
11         int age = scanner.nextInt();
12
13         System.out.println("姓名: " + name);
14         System.out.println("年龄: " + age);
15     }
16 }
```

Fence 117

十四、异常处理

1. 1. 什么是异常？

异常就是程序运行过程中出现的不正常情况。

例如：

```
1 | int a = 10 / 0;
```

Fence 118

会出现除零异常。

2. 2. 常见异常

异常	说明
ArithmeticException	算术异常
NullPointerException	空指针异常
ArrayIndexOutOfBoundsException	数组下标越界
ClassCastException	类型转换异常
NumberFormatException	数字格式异常

Table 8

3. 3. try...catch

```
1 | try {
2 |     int result = 10 / 0;
3 |     System.out.println(result);
4 | } catch (ArithmeticException e) {
5 |     System.out.println("出现算术异常");
6 | }
```

Fence 119

作用：

捕获异常，防止程序直接崩溃。

4. 4. 多个 catch

```
1  try {
2      int[] arr = {1, 2, 3};
3      System.out.println(arr[5]);
4  } catch (ArithmeticException e) {
5      System.out.println("算术异常");
6  } catch (ArrayIndexOutOfBoundsException e) {
7      System.out.println("数组下标越界");
8  } catch (Exception e) {
9      System.out.println("其他异常");
10 }
```

Fence 120

注意：

父类异常 `Exception` 要放在最后。

5. 5. finally

`finally` 中的代码一般都会执行。

```
1  try {
2      System.out.println(10 / 0);
3  } catch (Exception e) {
4      System.out.println("发生异常");
5  } finally {
6      System.out.println("最终执行");
7  }
```

Fence 121

6. 6. throws 抛出异常

```
1  public static void test() throws Exception {
2      System.out.println("可能出现异常的方法");
3  }
```

Fence 122

表示这个方法可能会产生异常，由调用者处理。

7. 7. throw 主动抛出异常

```
1  public static void checkAge(int age) {
2      if (age < 0) {
3          throw new RuntimeException("年龄不能为负数");
4      }
5  }
```

Fence 123

十五、集合框架

1. 1. 什么是集合？

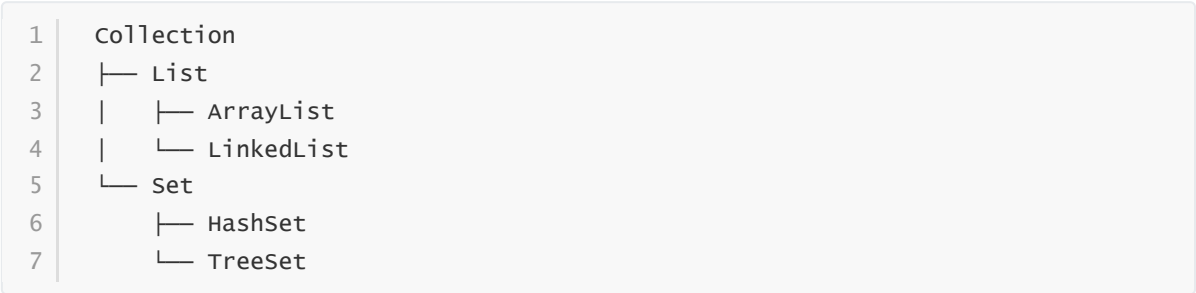
集合是用来存储多个对象的容器。

数组和集合的区别：

对比项	数组	集合
长度	固定	可变
存储内容	基本类型和对象	主要存对象
功能	较少	功能丰富

Table 9

2. 2. Collection 集合体系



Fence 124

3. 3. ArrayList

`ArrayList` 是最常用的集合之一。

特点：

- 有序
- 可重复
- 查询快
- 增删相对慢

示例：

```
1  import java.util.ArrayList;
2
3  public class Test {
4      public static void main(String[] args) {
5          ArrayList<String> list = new ArrayList<>();
6
7          list.add("Java");
8          list.add("Python");
9          list.add("C++");
```

```

10
11         System.out.println(list);
12         System.out.println(list.get(0));
13
14         list.remove("Python");
15
16         System.out.println(list.size());
17     }
18 }

```

Fence 125

4. 4. LinkedList

特点:

- 有序
- 可重复
- 增删快
- 查询相对慢

示例:

```

1  import java.util.LinkedList;
2
3  public class Test {
4      public static void main(String[] args) {
5          LinkedList<String> list = new LinkedList<>();
6
7          list.add("A");
8          list.add("B");
9          list.addFirst("Start");
10         list.addLast("End");
11
12         System.out.println(list);
13     }
14 }

```

Fence 126

5. 5. HashSet

特点:

- 无序
- 不重复
- 查询速度快

示例:

```

1  import java.util.HashSet;
2
3  public class Test {
4      public static void main(String[] args) {
5          HashSet<String> set = new HashSet<>();
6
7          set.add("Java");
8          set.add("Java");
9          set.add("Python");
10
11         System.out.println(set);
12     }
13 }

```

Fence 127

输出中只会会有一个 `Java`。

6.6. Map 集合

Map 用来存储键值对。

```

1  key -> value

```

Fence 128

例如：

```

1  学号 -> 学生姓名
2  001 -> 张三
3  002 -> 李四

```

Fence 129

7.7. HashMap

特点：

- 键不能重复
- 值可以重复
- 根据键查找值

示例：

```

1  import java.util.HashMap;
2
3  public class Test {
4      public static void main(String[] args) {
5          HashMap<String, String> map = new HashMap<>();
6
7          map.put("001", "张三");
8          map.put("002", "李四");
9          map.put("003", "王五");
10
11         System.out.println(map.get("001"));

```

```
12
13         map.remove("002");
14
15         System.out.println(map);
16     }
17 }
```

Fence 130

8. 8. 遍历 Map

```
1  HashMap<String, String> map = new HashMap<>();
2
3  map.put("001", "张三");
4  map.put("002", "李四");
5
6  for (String key : map.keySet()) {
7      System.out.println(key + " = " + map.get(key));
8  }
```

Fence 131

十六、Java 程序开发规范

1. 1. 命名规范

1.1 类名

使用大驼峰命名法。

```
1 Student
2 HelloWorld
3 UserService
```

Fence 132

1.2 变量名和方法名

使用小驼峰命名法。

```
1 studentName
2 getName()
3 setAge()
```

Fence 133

1.3 常量名

全部大写，单词之间用下划线。

```
1 MAX_VALUE
2 DEFAULT_SIZE
```

Fence 134

2. 2. 代码格式规范

推荐写法：

```
1 if (age >= 18) {
2     System.out.println("成年人");
3 } else {
4     System.out.println("未成年人");
5 }
```

Fence 135

不推荐：

```
1 if(age>=18){System.out.println("成年人");}
```

Fence 136

3. 3. 注释规范

代码关键位置要写注释。

```
1 // 判断年龄是否合法
2 if (age >= 0 && age <= 120) {
3     System.out.println("年龄合法");
4 }
```

Fence 137

4. 4. 常见初学者错误

4.1 1. 忘记写分号

错误：

```
1 int age = 18
```

Fence 138

正确：

```
1 int age = 18;
```

Fence 139

4.2 2. 类名和文件名不一致

文件名：

```
1 Test.java
```

Fence 140

代码：

```
1 public class HelloWorld {
2 }
```

Fence 141

这是错误的。

4.3 3. 数组下标越界

```
1 int[] arr = {1, 2, 3};
2 System.out.println(arr[3]);
```

Fence 142

错误原因：

数组下标是：

```
1 | 0, 1, 2
```

Fence 143

没有下标 3。

4.4 4. 空指针异常

```
1 | String str = null;  
2 | System.out.println(str.length());
```

Fence 144

错误原因：

`str` 是空，不能调用方法。

十七、综合案例：学生信息管理基础版

1. 1. 需求说明

实现一个简单的学生类，包含：

- 姓名
- 年龄
- 学号

并提供：

- 构造方法
- getter 和 setter
- 展示学生信息的方法

2. 2. Student 类

```
1  class Student {
2      private String name;
3      private int age;
4      private String id;
5
6      public Student() {
7      }
8
9      public Student(String name, int age, String id) {
10         this.name = name;
11         this.age = age;
12         this.id = id;
13     }
14
15     public String getName() {
16         return name;
17     }
18
19     public void setName(String name) {
20         this.name = name;
21     }
22
23     public int getAge() {
24         return age;
25     }
26
27     public void setAge(int age) {
28         if (age >= 0 && age <= 120) {
29             this.age = age;
30         } else {
31             System.out.println("年龄不合法");
32         }
33     }
34 }
```

```

34
35     public String getId() {
36         return id;
37     }
38
39     public void setId(String id) {
40         this.id = id;
41     }
42
43     public void showInfo() {
44         System.out.println("学号: " + id);
45         System.out.println("姓名: " + name);
46         System.out.println("年龄: " + age);
47     }
48 }

```

Fence 145

3. 3. 测试类

```

1     public class StudentTest {
2         public static void main(String[] args) {
3             Student stu = new Student("张三", 18, "001");
4             stu.showInfo();
5
6             System.out.println("-----");
7
8             stu.setName("李四");
9             stu.setAge(20);
10            stu.setId("002");
11
12            stu.showInfo();
13        }
14    }

```

Fence 146

4. 4. 输出结果

```

1     学号: 001
2     姓名: 张三
3     年龄: 18
4     -----
5     学号: 002
6     姓名: 李四
7     年龄: 20

```

Fence 147

十八、课堂练习

1. 练习一：变量与输出

要求：

定义姓名、年龄、成绩三个变量，并输出。

参考代码：

```
1  public class Practice01 {
2      public static void main(String[] args) {
3          String name = "张三";
4          int age = 18;
5          double score = 95.5;
6
7          System.out.println("姓名: " + name);
8          System.out.println("年龄: " + age);
9          System.out.println("成绩: " + score);
10     }
11 }
```

Fence 148

2. 练习二：判断成绩等级

要求：

根据分数输出等级：

- 90 分以上：优秀
- 80 到 89：良好
- 60 到 79：及格
- 60 以下：不及格

参考代码：

```
1  public class Practice02 {
2      public static void main(String[] args) {
3          int score = 85;
4
5          if (score >= 90) {
6              System.out.println("优秀");
7          } else if (score >= 80) {
8              System.out.println("良好");
9          } else if (score >= 60) {
10             System.out.println("及格");
11          } else {
12              System.out.println("不及格");
13          }
14     }
15 }
```

Fence 149

3. 练习三：数组求和

要求：

定义一个整数数组，求数组中所有元素的和。

参考代码：

```
1 public class Practice03 {
2     public static void main(String[] args) {
3         int[] arr = {10, 20, 30, 40, 50};
4
5         int sum = 0;
6
7         for (int i = 0; i < arr.length; i++) {
8             sum += arr[i];
9         }
10
11        System.out.println("数组元素和: " + sum);
12    }
13 }
```

Fence 150

4. 练习四：定义手机类

要求：

定义一个 `Phone` 类，包含：

- 品牌
- 价格
- 打电话方法
- 发短信方法

参考代码：

```
1 class Phone {
2     String brand;
3     double price;
4
5     public void call() {
6         System.out.println(brand + "正在打电话");
7     }
8
9     public void sendMessage() {
10        System.out.println(brand + "正在发短信");
11    }
12 }
13
14 public class PhoneTest {
15     public static void main(String[] args) {
16         Phone phone = new Phone();
17     }
```

```
18         phone.brand = "华为";  
19         phone.price = 3999;  
20  
21         phone.call();  
22         phone.sendMessage();  
23     }  
24 }
```

十九、知识总结

1. 1. Java 基础核心知识

1	Java 基础
2	├─ 基本语法
3	├─ 数据类型
4	├─ 运算符
5	├─ 流程控制
6	├─ 数组
7	├─ 方法
8	├─ 面向对象
9	├─ 异常处理
10	└─ 集合框架

Fence 152

2. 2. 面向对象三大特征

特征	说明
封装	隐藏属性，通过方法访问
继承	子类复用父类代码
多态	父类引用指向子类对象

Table 10

3. 3. 学习建议

- 1. 先掌握变量、if、循环、数组、方法。
- 2. 再理解类和对象。
- 3. 重点学习封装、继承、多态。
- 4. 多写代码，不要只看理论。
- 5. 遇到报错要学会阅读错误信息。

二十、课后作业

1. 作业一：计算最大值

定义一个方法，接收两个整数，返回较大的那个数。

2. 作业二：学生成绩统计

定义一个数组存储 5 个学生成绩，计算：

- 总分
 - 平均分
 - 最高分
 - 最低分
-

3. 作业三：员工类设计

定义一个 `Employee` 类，包含：

- 员工编号
- 员工姓名
- 工资

要求：

- 属性私有化
 - 提供 getter 和 setter
 - 提供显示员工信息的方法
-

4. 作业四：集合练习

使用 `ArrayList` 存储 5 个学生姓名，并完成：

- 添加学生
 - 删除学生
 - 遍历学生
 - 查询集合长度
-

结束语

Java 基础学习的重点不是背代码，而是理解程序的运行逻辑。

学习顺序建议：

1 | 语法基础 → 流程控制 → 数组 → 方法 → 类和对象 → 面向对象三大特征 → 集合和异常

Fence 153

只要多写、多调试、多总结，就能逐步掌握 Java 编程。